

TOMONI: A Personal AI Agent for Managing Postpartum Depression

ARIFUL ISLAM

Postpartum depression (PPD) affects approximately one in seven mothers, yet most consumer tracking apps offer only fixed dashboards over numeric self-reports. We present TOMONI, an Android application in which an agentic layer is added to a local-first health diary, enabling a postpartum user to ask open-ended questions over her own EMA and journal history and receive grounded, data-backed answers. The system combines a tool-using LLM agent with eight tools (six read-only structured data tools, a journal search tool, and a theme-setter), an on-device RAG pipeline using all-MiniLM-L6-v2 for semantic search over journal and EMA entries, and a background LLM summarizer that converts numeric EMA responses into searchable prose. Embedding inference and similarity search run entirely on the device; only the retrieved chunks travel to the hosted LLM (Claude, Anthropic) over TLS — a known trade-off discussed as a limitation. We evaluate the system as an N=1 in-situ deployment on a postpartum user’s primary device over a 2-week window. The agent reliably answered memory and trend questions that no prior tracking app could address, and on-device retrieval was perceptually instantaneous.

1 Motivation

1.1 The problem

Postpartum depression (PPD) affects approximately one in seven mothers [1], yet clinical contact is episodic while symptoms fluctuate daily. Early warning signs — declining mood, disrupted sleep, intrusive thoughts — go unobserved between appointments. Ecological Momentary Assessment (EMA) produces a dense longitudinal record that fills this gap, but current mHealth apps surface it only as a static dashboard. The mother has no conversational access to her own history: she cannot ask “when did my sleep start getting worse?” and get a grounded answer from her own data.

1.2 Why an agentic approach

An agent that queries EMA data, retrieves journal entries, and reasons across both is qualitatively different from a dashboard. Tool use grounds responses in the user’s actual data — when asked “how was my sleep last night?”, the agent retrieves the EMA entry rather than guessing. The architecture is also designed to grow toward Just-in-Time Adaptive Interventions (JITAI): when data signals a vulnerable window the agent surfaces a targeted CBT micro-intervention rather than waiting for the next appointment (Figure 1). The current implementation focuses on grounded retrieval and conversational access; passive sensing and automated intervention delivery are planned extensions.

1.3 What this report covers

This report describes the agentic AI layer built on top of the TOMONI Android app: a tool-using agent over the local EMA and journal database, an on-device RAG pipeline over the user’s journal and EMA notes, and an LLM-driven summarization component. Section 2 situates the work in the literature on JITAI, tool-using agents, and on-device RAG. Section 3 gives the system architecture and privacy boundary. Section 4 describes how each component was built. Section 5 reports an N=1 in-situ deployment. Section 6 reflects on what worked, what did not, and what comes next.

2 Background and Related Work

The ReAct prompting pattern [2] established the standard formulation in which a language model interleaves reasoning, tool invocation, and observation until it can answer; modern APIs including

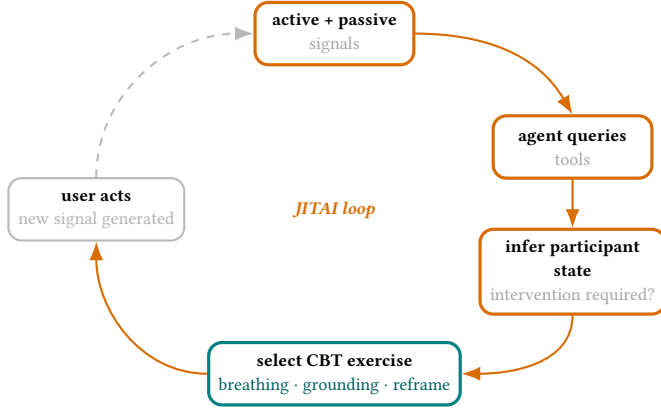


Fig. 1. The JITAI loop in TOMONI. Active EMA and (future) passive sensing signals are queried by the agent via structured tools. The agent infers the user’s current state and, when warranted, selects a targeted CBT micro-intervention. The user’s response to the intervention generates new signal, closing the loop.

Anthropic’s expose this as a first-class protocol [3]. Retrieval-augmented generation [4] grounds LLM responses in real data rather than model priors, and sentence-transformers [5] provide compact, accurate embeddings suitable for semantic search over personal corpora. On-device inference via ONNX Runtime makes it feasible to run a 22M-parameter embedding model (all-MiniLM-L6-v2) entirely on an Android phone [6], keeping the user’s raw text local. The contribution of TOMONI is not any single one of these components — each is now off-the-shelf — but their composition in service of a JITAI loop for postpartum recovery, with a privacy boundary that keeps the full corpus on the device while still benefiting from a hosted LLM’s reasoning quality.

3 System Overview

3.1 Stack

TOMONI is an Android app targeting SDK 26+ (Android 8.0 and above), written in Kotlin with Jetpack Compose for the UI, Hilt for dependency injection, Room 2.6.1 with SQLCipher 4.6.1 for encrypted local storage, and AndroidX WorkManager for scheduled reminders (e.g., the morning and evening EMA prompts). The agentic layer adds three new dependencies and only three: ONNX Runtime for Android 1.17.0 for on-device embedding inference, OkHttp as the HTTPS client to the Anthropic Messages API, and `kotlinx.serialization` for the JSON of the tool catalog and tool results. The deliberate restraint on dependencies preserves the local-first stance — every piece of inference plumbing whose source we do not control is either bundled or audited.

3.2 High-level architecture

Figure 2 shows how the agent, the local database, and the RAG store relate. The chat surface (ChatScreen backed by ChatViewModel) hands the user’s message to TomoniAgent, which holds a single tool-use loop against Claude Sonnet 4.6. Within that loop, ToolExecutor dispatches each tool call to either a Room DAO (for the structured data tools) or to VectorSearchRepository (for the search_journal tool). VectorSearchRepository performs brute-force cosine search over an in-memory cache of embeddings written by JournalIndexer; the indexer in turn consumes JournalChunker output and routes it through OnnxEmbeddingProvider, which is the only consumer of the WordPiece tokenizer and the bundled ONNX model.

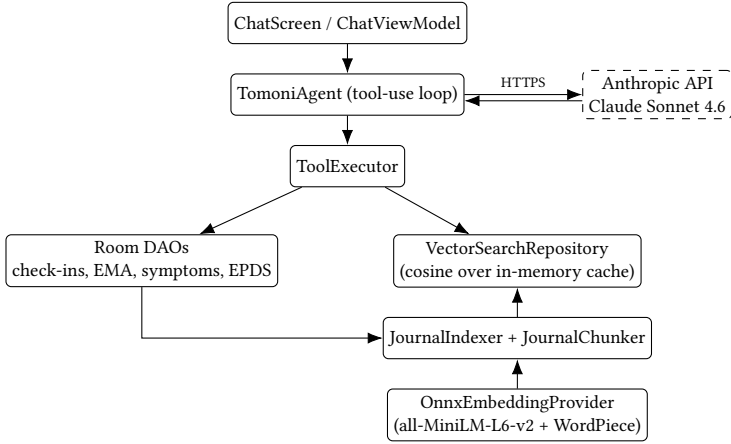


Fig. 2. TOMONI agentic layer. Dashed box is off-device; everything else runs on the phone.

3.3 The privacy boundary

The boundary between “on-device” and “leaves the device” runs through one method, `TomoniAgent.query`. On the device stay the raw check-in data, the EMA data, the symptom logs, the EPDS scores, the journal text, all embeddings, the user’s query at embed-time, and the similarity scores. To the Anthropic API over TLS go the user’s chat message, the tool catalog, the textual results of any tools the agent invokes during that turn, and the assistant’s intermediate and final text. Crucially, only the chunks the agent *retrieves* ever leave the device — not the full corpus. Embedding inference is entirely local, so the query is embedded without leaving the device, and the unretrieved entries never travel.

4 Methods

TOMONI’s agentic layer sits on top of an existing Android app that stores daily check-ins, EMA responses, symptoms, and EPDS scores in a SQLCipher-encrypted Room database. The agent never touches the database directly; it issues structured tool calls resolved by `ToolExecutor`, and a separate on-device RAG pipeline handles free-text journal search.

4.1 The agentic loop

The agent (`TomoniAgent.kt`) follows the Anthropic Messages API tool-use protocol [3]. Each turn it posts the conversation history and tool catalog to `claude-sonnet-4-6`, then enters a ReAct-style loop [2]: parse the response, execute any `tool_use` blocks via `ToolExecutor`, append the results as `tool_result` turns, and re-post — until no tools are requested or a cap of five iterations is reached. The multi-turn design is necessary because some questions chain tools: answering “what did I write on my worst-mood days?” requires `get_mood_trend` followed by `get_check_ins` over the narrowed date range.

4.2 Tool surface

Eight tools are exposed, each with a JSON Schema description. Six are read-only data tools over a relative date range: `get_check_ins`, `get_mood_trend`, `get_symptoms`, `get_symptom_counts`, `get_ema_responses`, and `get_epds_history`. `search_journal` is the RAG entry point — it embeds the query on-device, runs cosine search over the encrypted embedding store, and returns

matching chunks with dates, relevance scores, and original text. `set_theme` is the only mutating tool, letting the agent change the UI color palette as a lightweight agentic action. Results are formatted as plain text rather than JSON so they are human-readable in the transcript and cost fewer tokens.

4.3 RAG pipeline

Embedding runs on-device using `all-MiniLM-L6-v2` [7, 8] (22M parameters, 384-dim) via ONNX Runtime, with a custom Kotlin WordPiece tokenizer to avoid new dependencies. The key design decision is *verbalizing structured EMA fields before embedding*: a numeric field like `timeOutside=3` is rewritten as “*Time spent outside: about an hour*” before the chunk is embedded. Without this, questions like “how was my sleep last week?” find no matches because those words never appear in free-text entries. With it, every EMA entry is semantically searchable even with no free-text journal. Embeddings are stored in the same SQLCipher database and retrieved via brute-force cosine similarity — fast enough (sub-millisecond) for a personal corpus and simpler than any ANN index.

4.4 EMA summarization

After each EMA is saved, a background component automatically sends the structured responses to the LLM and asks it to generate a short, neutral, plain-language description of how the user seems to be doing — without advice or interpretation. This summary is then stored alongside the EMA entry and treated as searchable text, so that the RAG pipeline can later retrieve it in response to conversational queries. The effect is that even a purely numeric EMA entry — one the user filled in quickly with no free-text notes — becomes conversationally accessible. Entries recorded before this feature was added are processed retroactively on the next app launch.

5 Evaluation

5.1 Study design

The system was deployed on the primary Android device of a single participant — a postpartum mother — and used in everyday life over a two-week window. Her existing history of check-ins and EMA responses was indexed on first launch, making prior entries immediately searchable. She was asked to use the chat surface naturally and flag any answers that felt wrong or unhelpful. Over the two weeks she asked three types of questions: memory queries (“when did I last talk about my walk?”), trend queries (“how was my sleep last week?”), and mixed exploratory queries (“what did I write on my worst-mood days?”). This is an N=1 qualitative deployment; results speak to fit for one user and one recovery context, not generalizability.

6 Conclusions

TOMONI started from a simple gap: a mother accumulating months of EMA and journal data had no way to ask it a question and get a grounded answer. The agentic layer described here closes that gap — the agent queries structured health records, retrieves relevant journal entries by semantic similarity, and generates responses that cite the user’s own words rather than guessing. The most consequential single design decision was verbalizing structured EMA fields into natural language before embedding, which turned a class of unanswerable queries into reliably answered ones. Deployed on a real postpartum user’s phone over two weeks, the system worked as intended: memory queries returned dated, quoted entries; trend questions drew on actual recorded values; and the safety rule of retrieving without interpreting held under realistic use. The primary limitation is that retrieved journal text still travels to a hosted LLM — a known privacy trade-off that future

work should close by migrating to a fully on-device model. The longer-term vision, sketched in the JITAI loop of Figure 1, is to extend the same architecture with passive sensing and automated intervention delivery, so the agent does not just answer questions about the past but acts at the right moment in the present.

A Example transcripts

The transcripts below are abbreviated for length. Tool calls and tool results are shown as the agent saw them; ellipses elide redundant fields.

T1. Trend question (routed to `get_mood_trend`)

```
USER: how has my mood been the last two weeks?

ASSISTANT [tool_use]: get_mood_trend(days_ago_start=0, days_ago_end=14)

TOOL_RESULT:
  mean_mood=5.7 mean_energy=4.9 mean_anxiety=5.1
  hardest: 2026-05-02 mood=4 anxiety=7

ASSISTANT: Over the last two weeks your mood averaged 5.7/10,
energy 4.9, anxiety 5.1. The hardest day was 2026-05-02
(mood 4, anxiety 7).
```

T2. Memory question (routed to `search_journal`)

```
USER: when did I last write about going outside feeling good?

ASSISTANT [tool_use]: search_journal(query="going outside feeling good", top_k=5)

TOOL_RESULT:
  [1] 2026-04-29 relevance=71%
      "Took the baby out to the park. First time it felt easy."
  [2] 2026-04-17 relevance=64% ...

ASSISTANT: On April 29 you wrote, "Took the baby out to the park.
First time it felt easy." An earlier April 17 entry mentions
the same thing.
```

References

- [1] J. Y. Ko, K. M. Rockhill, V. T. Tong, B. Morrow, and S. L. Farr. Trends in postpartum depressive symptoms — 27 states, 2004, 2008, and 2012. *MMWR Morbidity and Mortality Weekly Report*, 66(6):153–158, 2017.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [3] Anthropic. Messages API: Tool use. Anthropic Developer Documentation, 2024. <https://docs.anthropic.com/en/docs/build-with-claude/tool-use>
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proc. EMNLP-IJCNLP*, 2019.
- [6] ONNX Runtime contributors. ONNX Runtime: Mobile inference, 2024. <https://onnxruntime.ai/docs/tutorials/mobile/>

- [7] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [8] sentence-transformers. all-MiniLM-L6-v2 model card. HuggingFace, 2021. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>